

State and Action Spaces for Movement Control

Controllable 2D Point Mass

Planar 2-link Robot

Underactuated Planar 2-link Robot

Humanoid Robot / Simple Human Model

Previously...

- simplest class of MDP

- graph: discrete s, a ; deterministic f, r ; model known; “Dynamic Programming”

- $\pi(s) \quad G_\pi(s) \quad V_\pi(s)$

- policy iteration, value iteration, implicit policy:

$$V(s) \leftarrow \max_a \{ r(s, a) + \gamma V(s') \}$$

$$\pi(s) = \arg \max_a \{ r(s, a) + \gamma V(s') \}$$

- ☹ cannot learn from episodic data:

$$\{ (s, a, r, s'), (s', a, r, s''), (s'', a, r, s'''), \dots \}$$

i.e., when the model is missing; both algos need to loop over each state

Loop for each $s \in \mathcal{S}$:

Previously (continued)

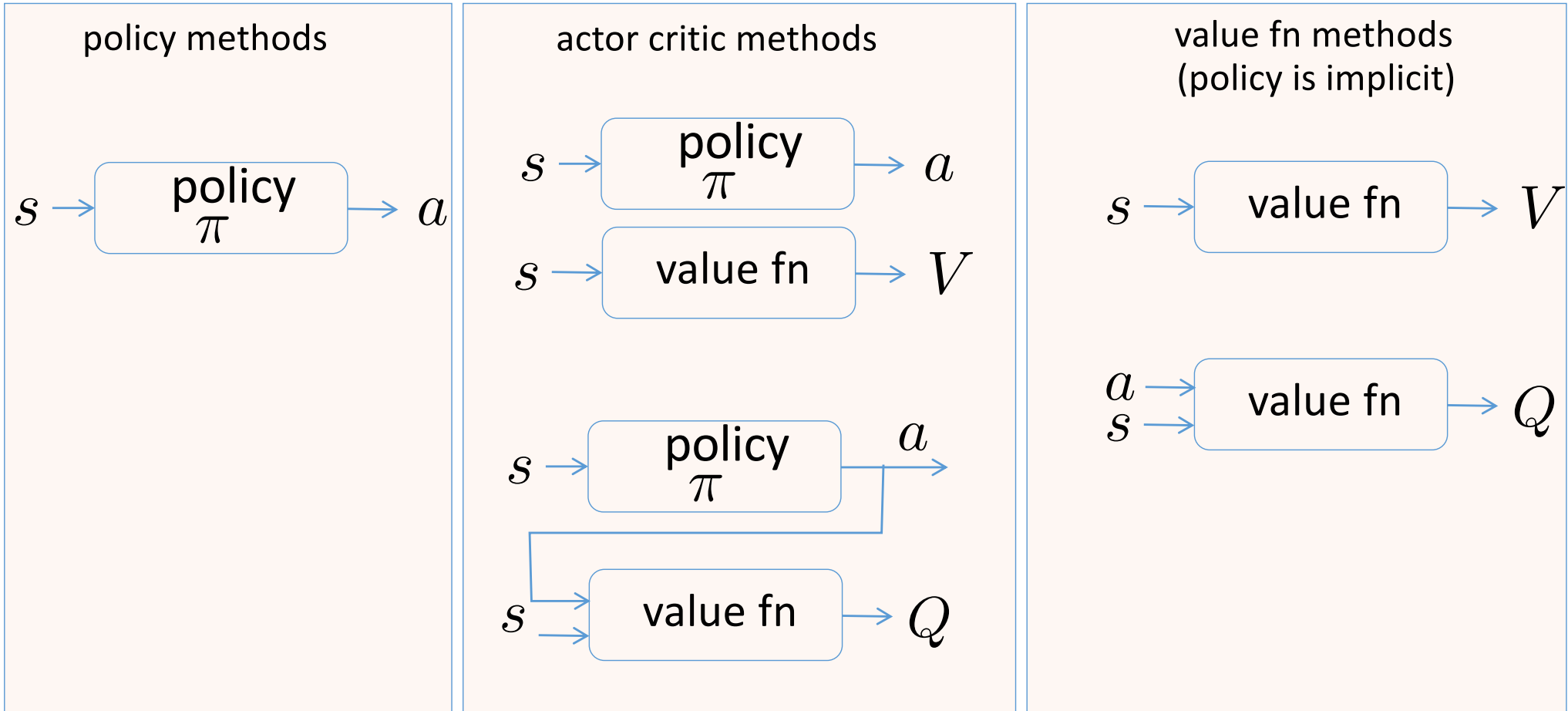
- $Q(s,a)$ state action value function
 - implicit policy does not require knowing the model ☺

$$\pi(s) = \arg \max_a (Q(s, a))$$

- can potentially learn from streams of episodic data ☺

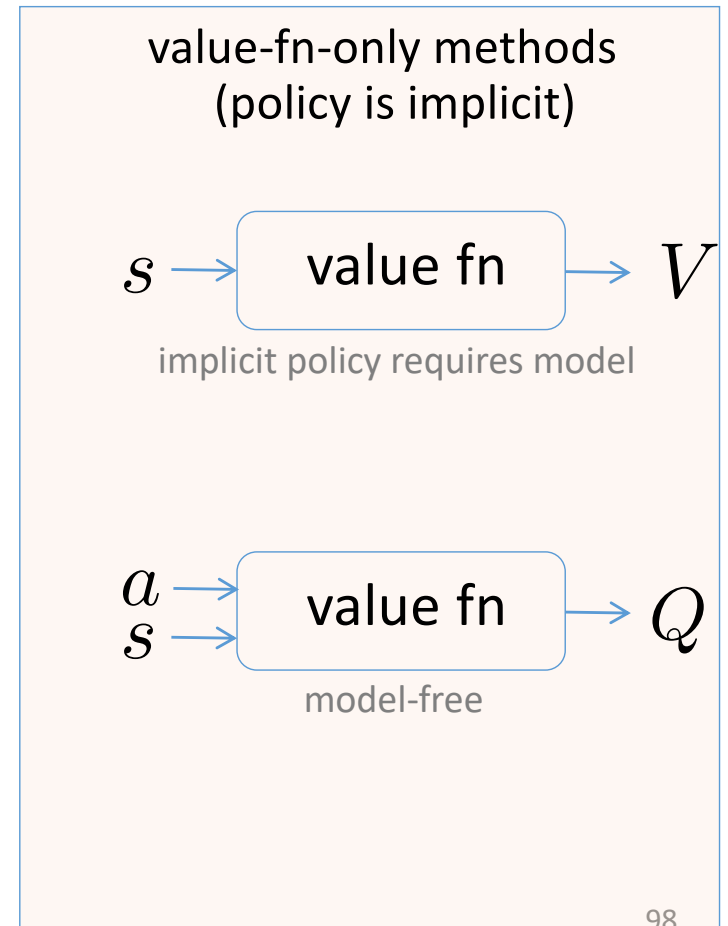
$$Q_{k+1}(s, a) = r + \gamma \max_{a'} (Q_k(s', a'))$$

Deep RL: key building blocks



Q-learning

(off-policy TD version)



Temporal Difference Learning

(at the heart of nearly all practical RL algorithms)

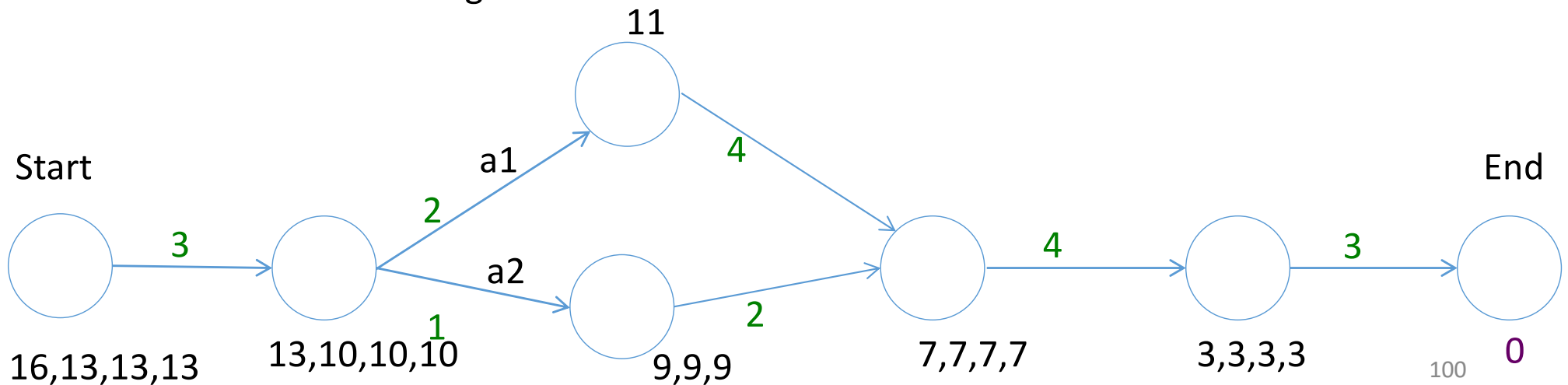
- TD methods learn directly from episodes of experience
- TD is model-free: no knowledge of MDP transitions / rewards
- TD learns from incomplete episodes, by bootstrapping
- TD updates a guess towards a guess

[Silver]

Monte Carlo Return Estimation

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Record the actual future returns seen,
from numerous examples where state s was visited,
and then take the average.



First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

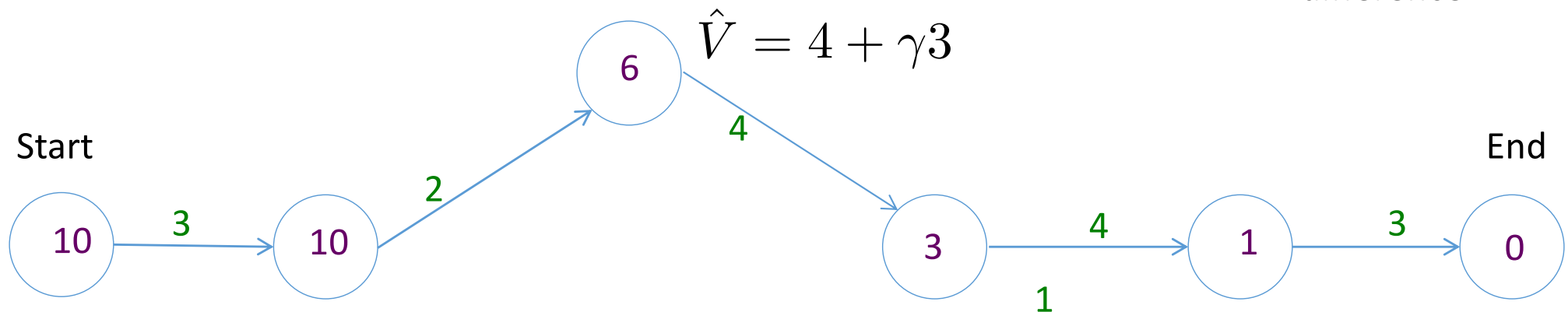
$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Alternative Approach: Temporal-Difference Return Estimation: TD(0)

- Given a policy, estimate its value function from episodic data

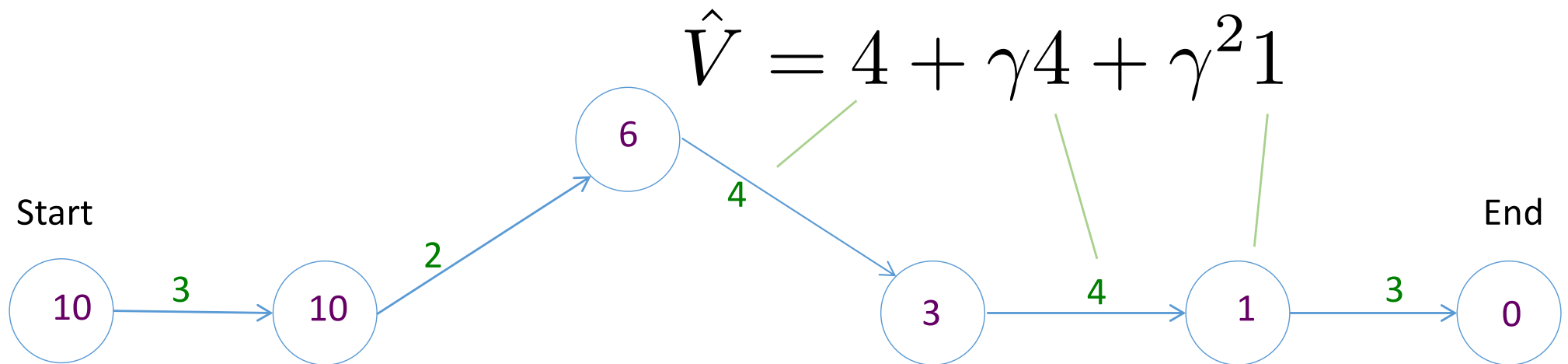
$$V(S_t) \leftarrow V(S_t) + \alpha(\underbrace{r_{t+1} + \gamma V(S_{t+1})}_{\text{target}} - V(S_t))$$

δ temporal difference



if model is known: $V(s) = \sum_{s', r} p(s', r | s, a) [r(s, a) + \gamma V(s')]$

n-step Temporal Differences



n-step Temporal Differences, all the way to Monte Carlo estimation

Let's label the estimated return following n steps as $G_t^{(n)}$, $n = 1, \dots, \infty$, then:

n	G_t	Notes
$n = 1$	$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$	TD learning
$n = 2$	$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$	
...		
$n = n$	$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$	
...		
$n = \infty$	$G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T + \gamma^{T-t} V(S_T)$	MC estimation

The generalized n -step TD learning still has the same form for updating the value function:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$$

[lillianweng]

TD(λ)

Which value of n is best?

- small n : simple, efficient

- large n : more accurate – sees more real rewards

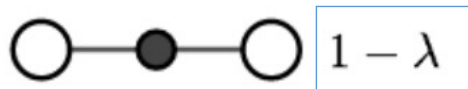
→ just use a weighted mix?

relative
weight

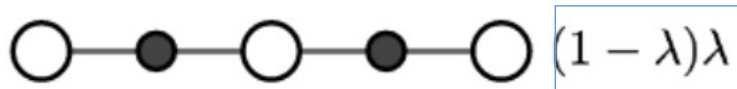
At time step t , TD(λ)

actual weights, normalized to sum to 1

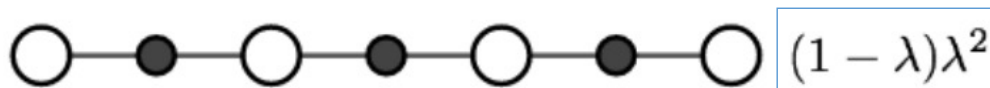
1



λ



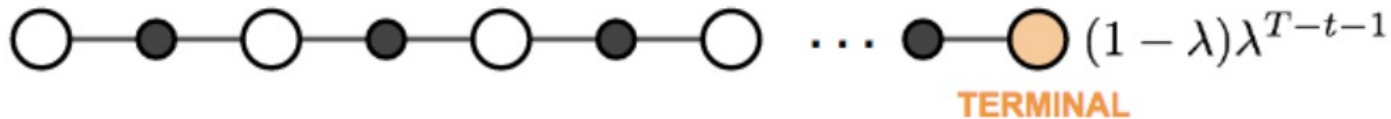
λ^2



...

...

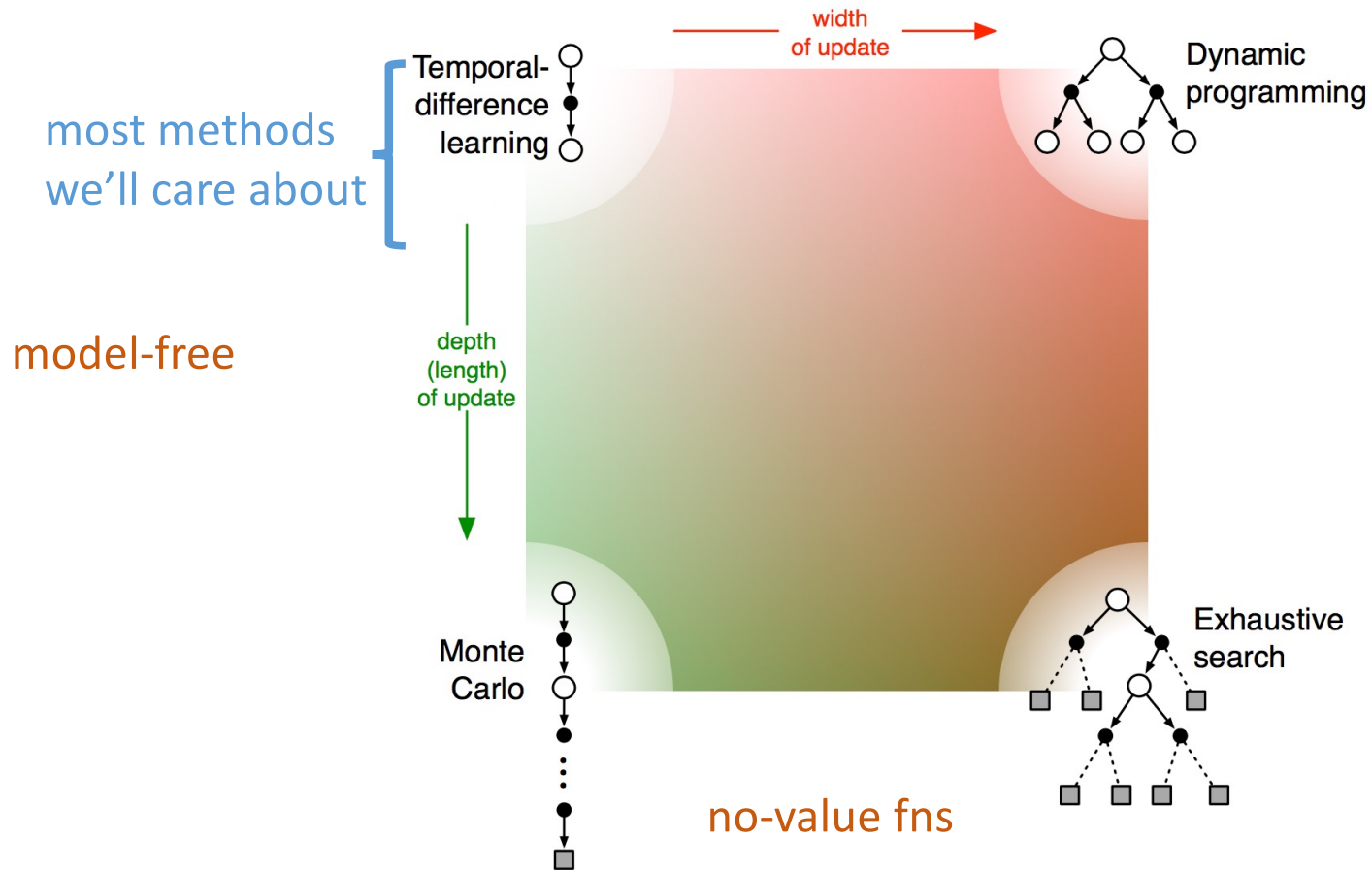
————



$\frac{1}{1 - \lambda}$

total

bootstrapping, i.e., value-functions

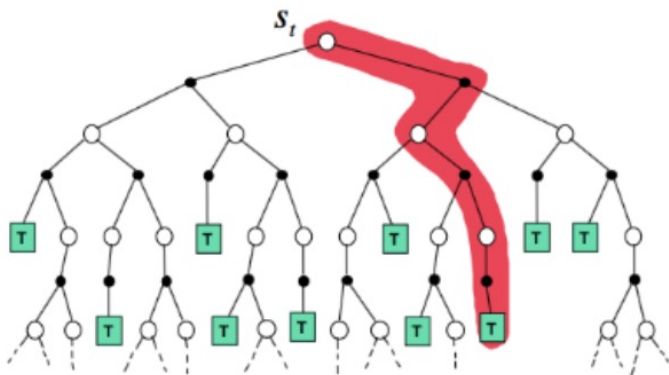


[Sutton, p190]

Another view of the corners of the design space

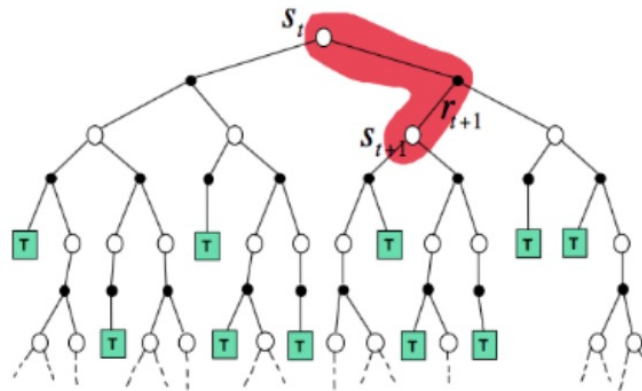
Monte-Carlo

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



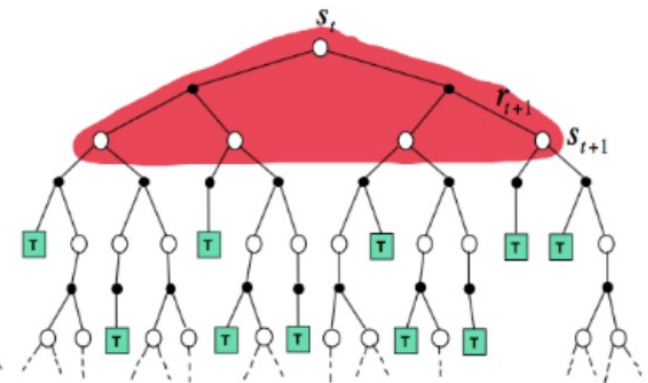
Temporal-Difference

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_\pi [R_{t+1} + \gamma V(S_{t+1})]$$



[Silver/Wang]

Updating State-Value Functions, $V_\pi(s)$

actual returns $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
 $G_t = r_{t+1} + \gamma G_{t+1}$

expected returns $V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$

$V_\pi(s) = G_\pi(s)$ for deterministic everything

$V(s) = \sum_{s', r} p(s', r | s, a) [r(s, a) + \gamma V(s')]$ if model is known

Model-free, stochastic dynamics, using observed trajectories, with TD-updates:

$$V(S_t) \leftarrow V(S_t) + \alpha(\underbrace{r_{t+1} + \gamma V(S_{t+1})}_{\text{target}} - V(S_t))$$

$$V(S_t) \leftarrow V(S_t) + \alpha(\hat{V}(S_t) - V(S_t)) \quad \delta$$

$$V(S_t) \leftarrow (1 - \alpha)V(S_t) + \alpha\hat{V}(S_t)$$

old new

note: these trajectories need to come from following the current policy.

Updating the Policy using $V(s)$

model known:

$$\pi(s) \leftarrow \arg \max \sum [r(s, a) + \gamma V(s')]$$

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

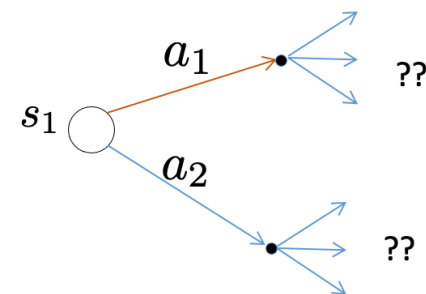
deterministic dynamics

stochastic dynamics

General case, i.e., model-free, stochastic dynamics:

? need to know the next-state and reward

? exploration: how to visit all states?



Updating Action-Value Functions, $Q(s,a)$

expected returns

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, a_t = a]$$

following policy π afterwards;
we'll assume the optimal policy

$$Q(s, a) = r + \gamma \max_{a'} Q(s, a')$$

deterministic dynamics

$$Q(s, a) = \sum_{s', r} p(s', r) [r(s, a) + \gamma \max_{a'} Q(s, a')]$$

stochastic dynamics

Model-free, stochastic dynamics, using observed trajectories, with TD-updates:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [\underbrace{r_{t+1} + \gamma \max_a Q(s_{t+1}, a)}_{\text{target}} - Q(s_t, a_t)]$$

target

δ

note: can follow any trajectories (“off policy”), provided all (s,a) are visited repeatedly¹¹⁰

Policy Using $Q(s,a)$

- nothing to update – the policy is implicit!

$$\pi(s) = \arg \max_a (Q(s, a))$$

- should somehow generate trajectories that visit all states, and try all actions in each state
- even as the $Q(s,a)$ changes, “old” episode data can be reused! e.g., $\{s,a,r,s'\}$ tuples. Store these in an “experience replay” buffer. Eventual NN implementation: use this to generate IID updates.

ϵ -Greedy Exploration (for discrete actions)

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

$$\pi(a|s) = \begin{cases} \epsilon/m + 1 - \epsilon & \text{if } a^* = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a) \\ \epsilon/m & \text{otherwise} \end{cases} \quad \text{for } m \text{ discrete actions}$$

[Silver]

Q Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

More on Q-learning

- provably converges in the tabular setting (discrete states & actions)
as long as all state-action pairs continue to be updated

Maximization Bias

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

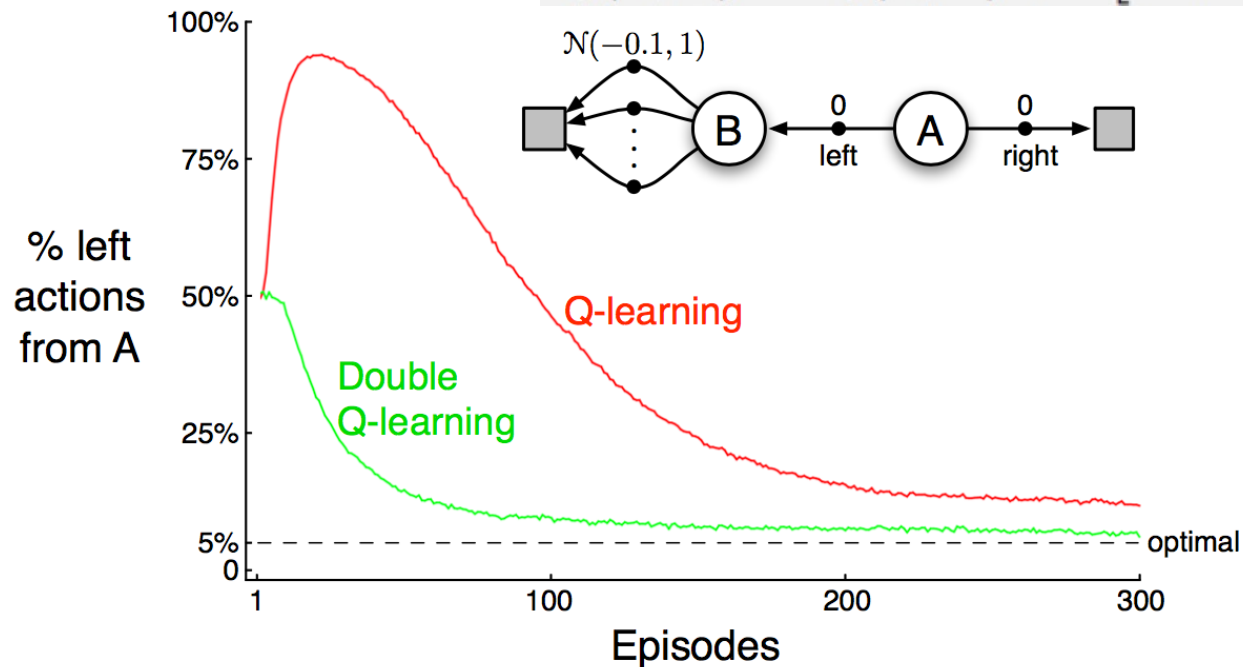


Figure 6.5: Comparison of Q-learning and Double Q-learning on a simple episodic MDP (shown inset). Q-learning initially learns to take the left action much more often than the right action, and always takes it significantly more often than the 5% minimum probability enforced by ϵ -greedy action selection with $\epsilon = 0.1$. In contrast, Double Q-learning is essentially unaffected by maximization bias. These data are averaged over 10,000 runs. The initial action-value estimates were zero. Any ties in ϵ -greedy action selection were broken randomly.

[Sutton p134] ¹¹⁵

A fix: Double Q Learning

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, such that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using the policy ε -greedy in $Q_1 + Q_2$

 Take action A , observe R, S'

 With 0.5 probability:

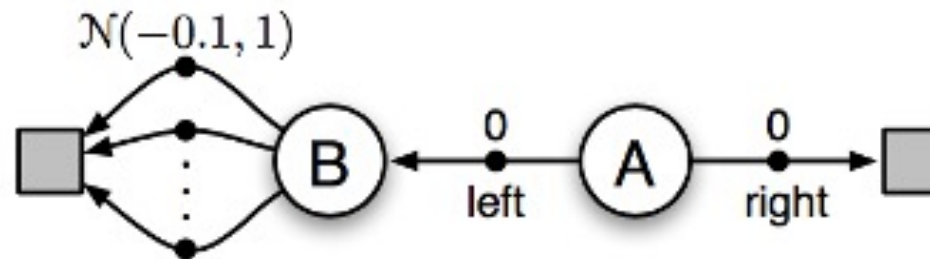
$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha \left(R + \gamma Q_2(S', \arg \max_a Q_1(S', a)) - Q_1(S, A) \right)$$

 else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha \left(R + \gamma Q_1(S', \arg \max_a Q_2(S', a)) - Q_2(S, A) \right)$$

$S \leftarrow S'$

 until S is terminal



Q_1 Q_2

$$\hat{Q}_1 = 0 + \gamma Q_2(s', a_1^*) \quad Q_1 \text{ target}$$

a_1^*	0.5	-0.2
	-0.6	0.4
	0.2	-0.3

more difficult for a “lucky” reward
to propagate

11.3 The Deadly Triad [Sutton, p264]

Our discussion so far can be summarized by saying that the danger of instability and divergence arises whenever we combine all of the following three elements, making up what we call *the deadly triad*:

Function approximation A powerful, scalable way of generalizing from a state space much larger than the memory and computational resources (e.g., linear function approximation or ANNs).

Bootstrapping Update targets that include existing estimates (as in dynamic programming or TD methods) rather than relying exclusively on actual rewards and complete returns (as in MC methods).

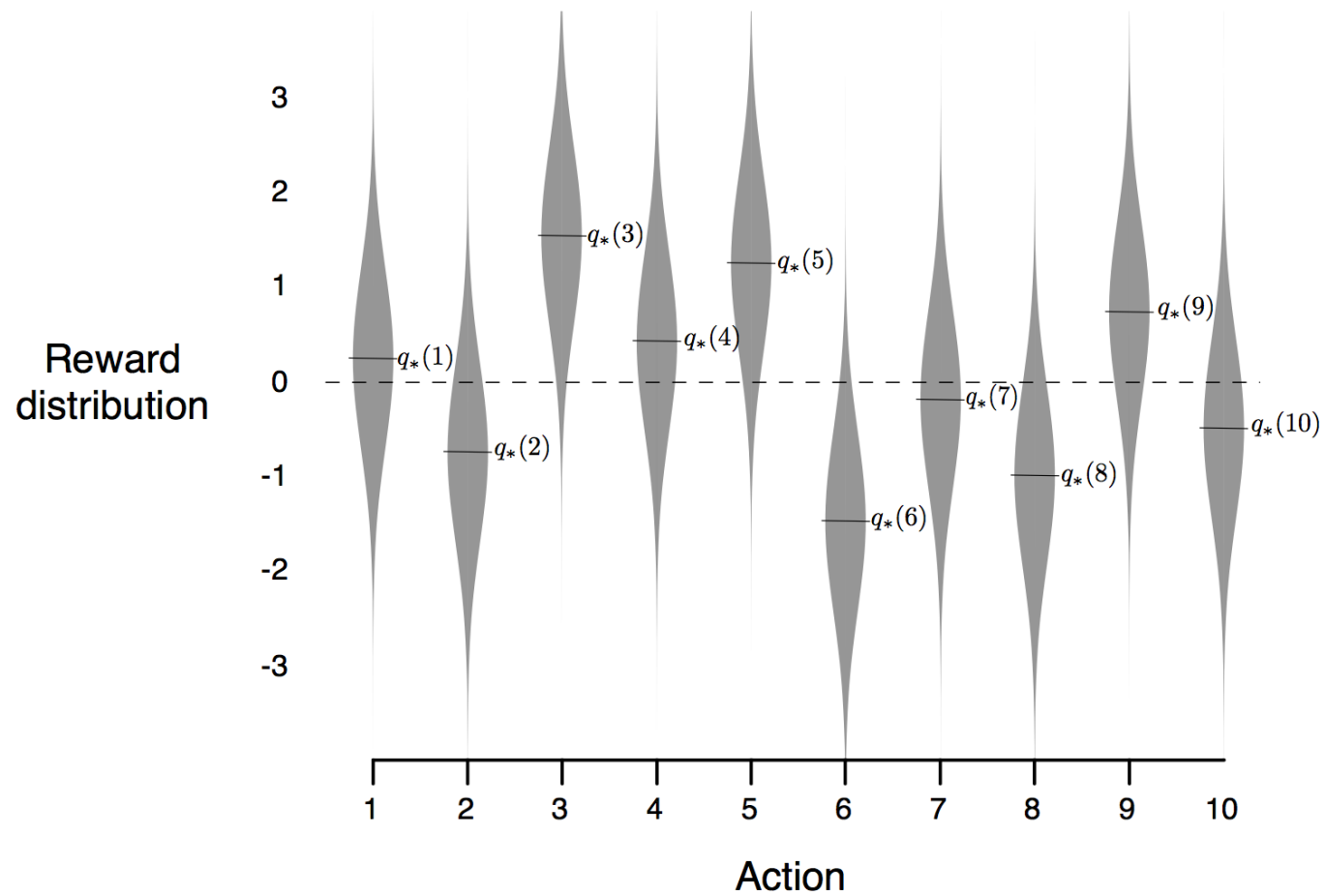
Off-policy training Training on a distribution of transitions other than that produced by the target policy. Sweeping through the state space and updating all states uniformly, as in dynamic programming, does not respect the target policy and is an example of off-policy training.

[Allows learning of multiple policies
from a single stream of data]

More about value function updates...

- fixed and changing steps sizes; connection to Bandits
- equivalence to a loss function update

Bandits



[Sutton, p28]

Q-functions for bandits

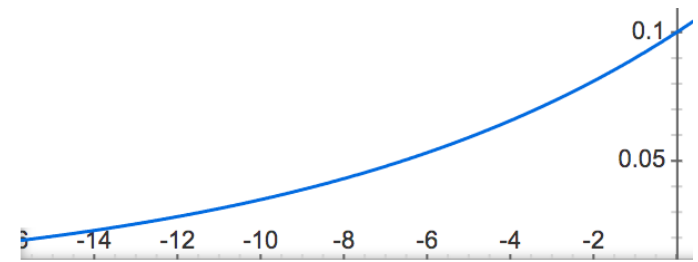
$$\begin{aligned}Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} (R_n + (n-1)Q_n) \quad \leftarrow Q_n \doteq \frac{R_1 + R_2 + \dots + R_{n-1}}{n-1} \\&= \frac{1}{n} (R_n + nQ_n - Q_n) \\&= Q_n + \frac{1}{n} [R_n - Q_n], \quad \longleftrightarrow V(S_t) \leftarrow V(S_t) + \alpha(\hat{V}(S_t) - V(S_t))\end{aligned}$$

decreasing “step size” over time constant “step size”

Updating using a constant step size

This gives more weight to recent rewards,
which is useful when tracking a non-stationary problem.

$$\begin{aligned}Q_{n+1} &= Q_n + \alpha [R_n - Q_n] \\&= \alpha R_n + (1 - \alpha) Q_n \\&= \alpha R_n + (1 - \alpha) [\alpha R_{n-1} + (1 - \alpha) Q_{n-1}] \\&= \alpha R_n + (1 - \alpha) \alpha R_{n-1} + (1 - \alpha)^2 Q_{n-1} \\&= \alpha R_n + (1 - \alpha) \alpha R_{n-1} + (1 - \alpha)^2 \alpha R_{n-2} + \\&\quad \dots + (1 - \alpha)^{n-1} \alpha R_1 + (1 - \alpha)^n Q_1 \\&= (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i.\end{aligned}$$



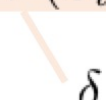
[Sutton, p32]

note: sum of weights = 1

Off-policy vs On-policy learning

- On-policy: Use the deterministic outcomes or samples from the target policy to train the algorithm.
- Off-policy: Training on a distribution of transitions or episodes produced by a different behavior policy rather than that produced by the target policy. For example, the actions could come from another agent.
- Offline / Batch / “fully off policy” RL: learn from a fixed static dataset
- Q-learning allows for off-policy learning

Loss-function view of a value function update

$$V(S_t) \leftarrow V(S_t) + \alpha(\underbrace{r_{t+1} + \gamma V(S_{t+1})}_{\text{target}} - V(S_t))$$


A diagram with an orange arrow pointing from the word 'delta' to the difference between the target and the current value in the equation above.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$$\min_V f(V) = (\hat{V} - V)^2$$

Bellman error squared

$$\frac{\partial f}{\partial V} = -2(\hat{V} - V)$$

$$V \leftarrow V - \frac{\alpha}{2} \frac{\partial f}{\partial V}$$

take a step size of 0.5 alpha, downhill

$$V \leftarrow V + \alpha(\hat{V} - V)$$

and similarly for Q(s,a)

Loss-function view of a value function update